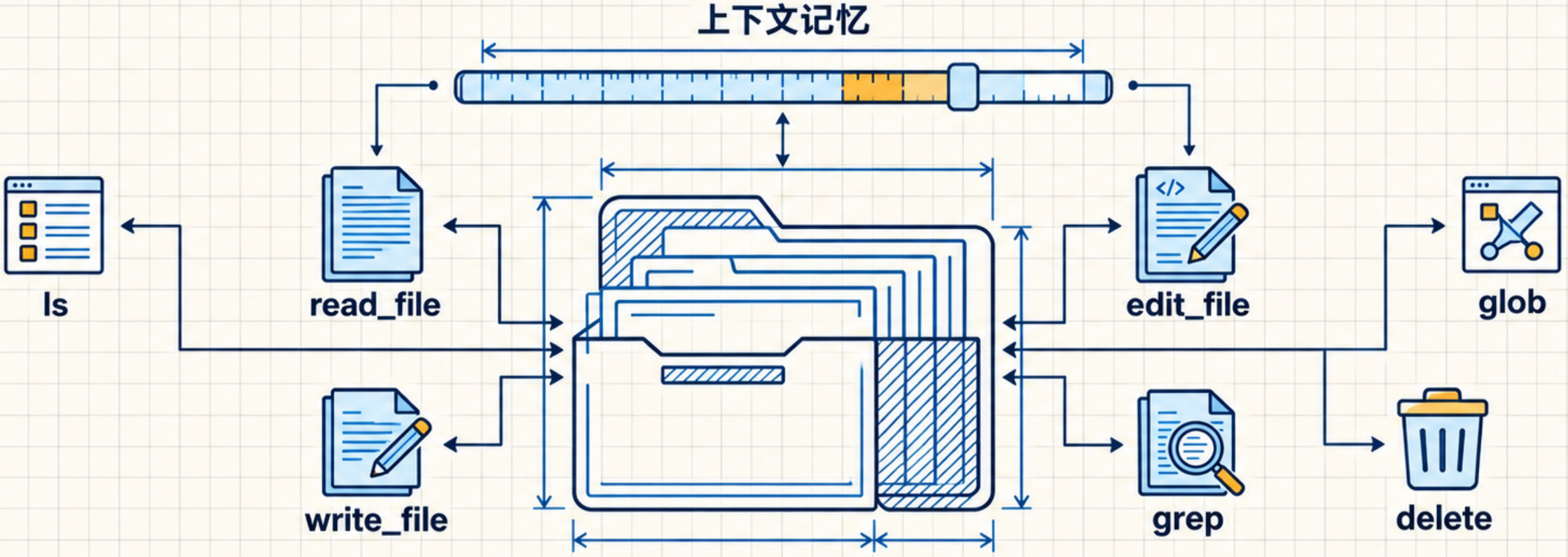
Deep Agents 实战

v0.7 内容校准版

第 3 章 · 上篇：虚拟文件系统与上下文管理



把资料放进工作空间，让上下文只保留当前所需。

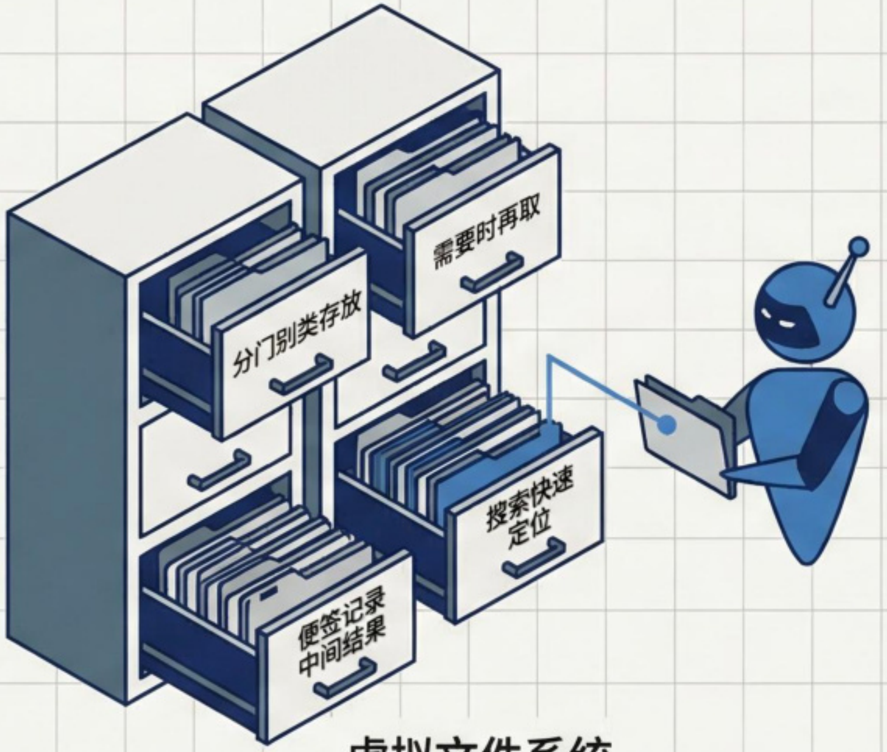
为什么用文件系统管理上下文?

从"全塞进 Prompt"到"按需读取"



传统 Prompt Stuffing

- 传统做法：所有信息直接塞进 prompt，对话历史不断膨胀



虚拟文件系统

- Deep Agents 方案：给 Agent 一个文件系统
- 像人一样工作：分门别类存放、需要时再取、搜索快速定位、便签记录中间结果

七大文件系统工具

从查找到读写，再到删除



ls

列出文件与元信息



read_file

分页读取；支持多模态



write_file

创建或完整覆盖文件



edit_file

精确字符串替换



delete

删除文件或目录



glob

按文件名模式查找



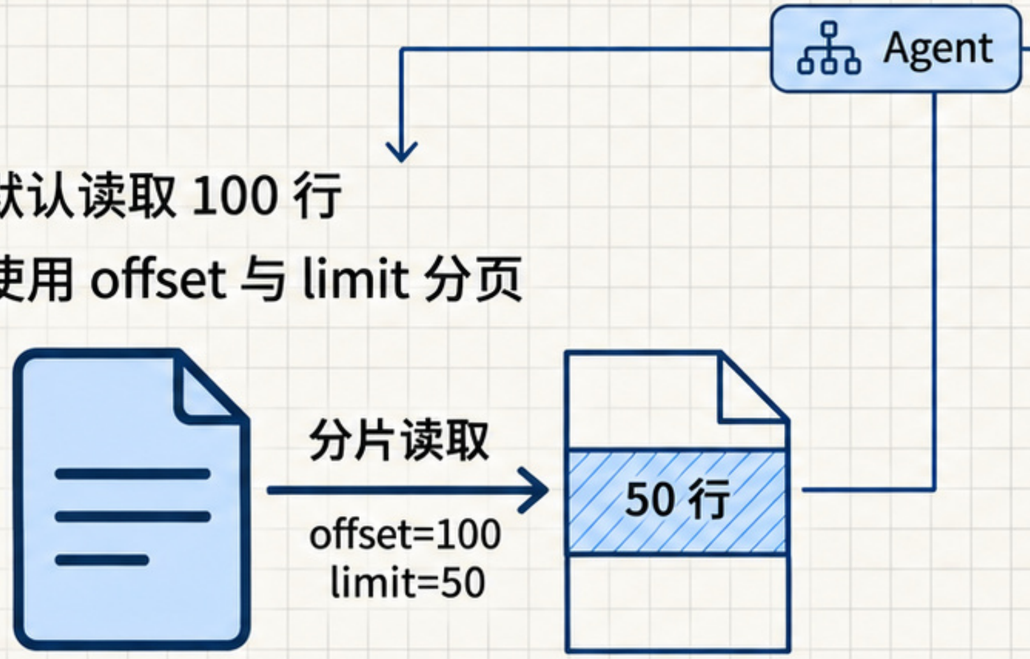
grep

字面量搜索，不是正则匹配

⚠ | 搜索结果可能截断；写入与删除都需要权限边界。

read_file：按需读取，不塞满上下文

文本分页 + 多模态内容块



- 默认读取 100 行
- 使用 offset 与 limit 分页
- 根据返回的下一页位置继续读取

```
read_file("/workspace/report.md", offset=100, limit=50)
```

工具调用示意；不是可直接导入的独立 Python 函数。

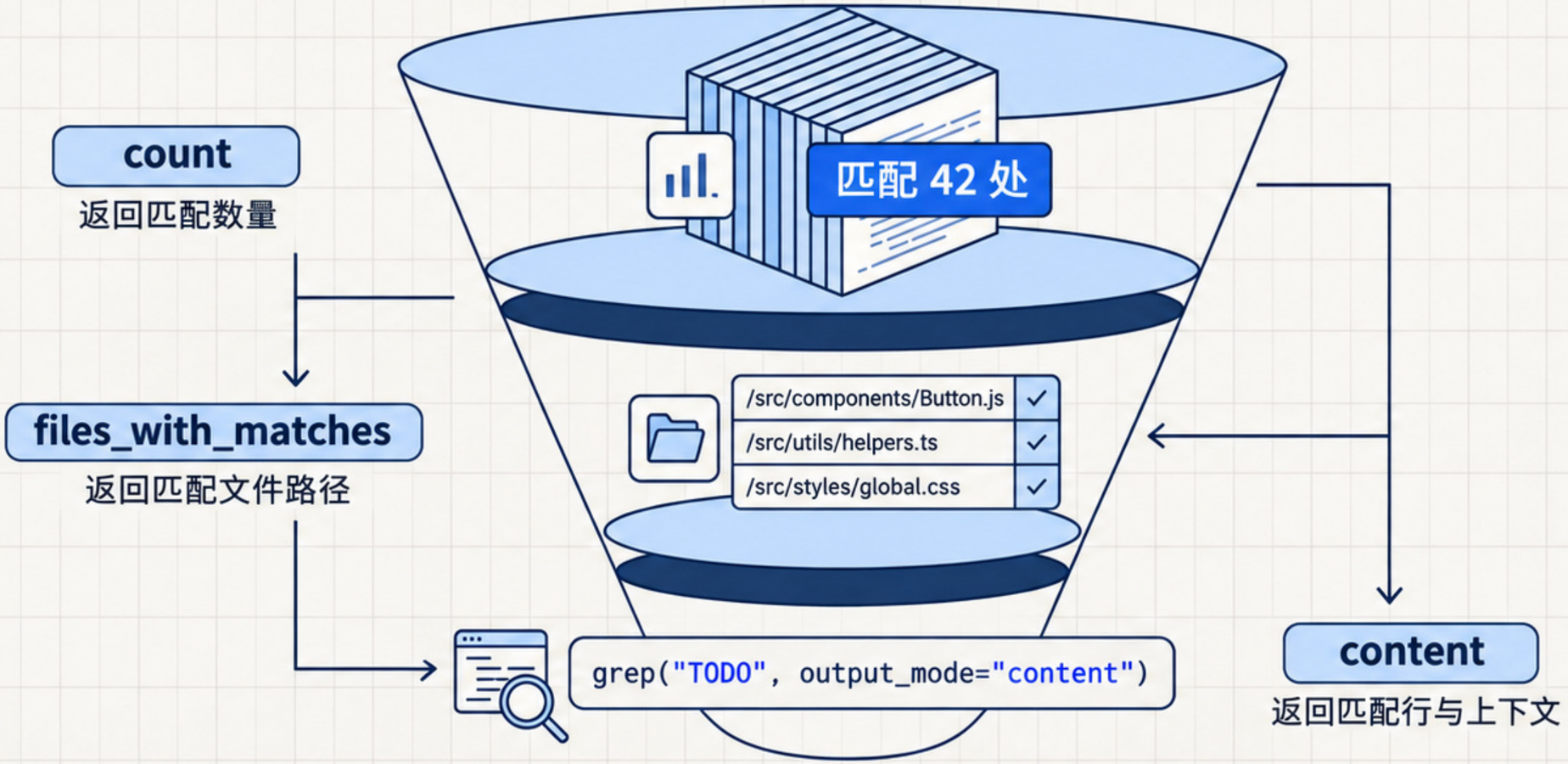
转换为多模态内容块

图片	.png .jpg .jpeg .gif .webp .heic .heif
视频	.mp4 .mpeg .mov .avi .flv .webm .wmv .3gpp
音频	.wav .mp3 .aiff .aac .ogg .flac
PDF 与 PPT	.pdf .ppt .pptx

实际能否处理该格式，取决于模型与提供商支持。

grep：三种输出模式

字面量匹配，从定位到查看

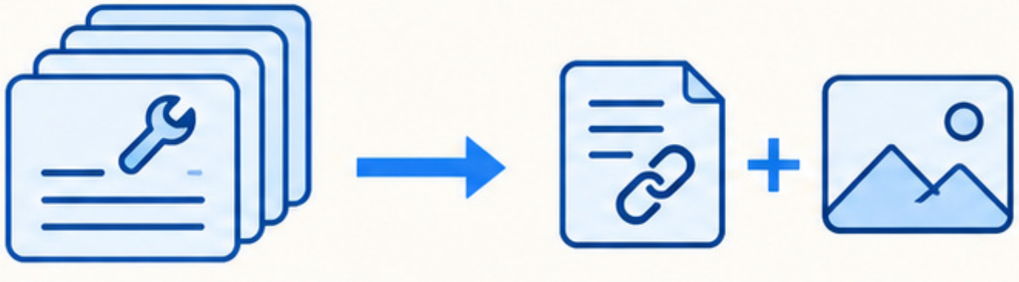


有结果，不代表已遍历全部 | grep / glob 可能返回有效但不完整的结果 `truncated=True` | grep 默认最多返回 1000 条匹配；应缩小范围后继续检索。

上下文自动管理

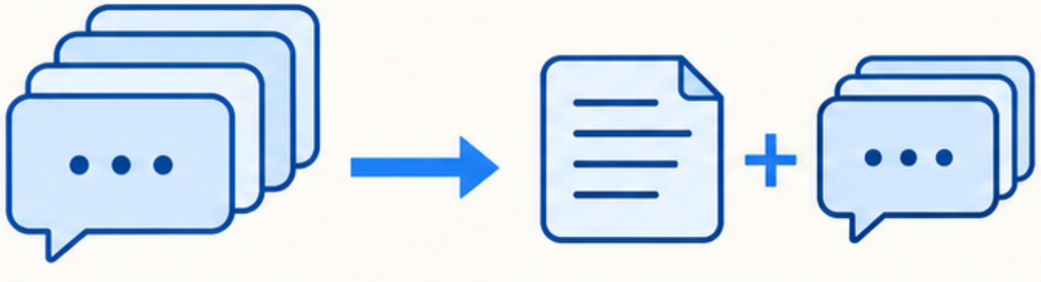
两道防线，解决两种不同的膨胀

大型工具内容卸载



默认阈值 20,000 tokens，可配置

对话历史总结



已知模型窗口时，默认在约 85% 触发

⚠ 未知窗口时，使用固定 token 阈值回退

两类阈值都可调整；不要把它们误读成同一条容量刻度。

第一道防线：大型工具内容卸载

把长内容移到文件，给模型留下索引

Step 1: 触发条件

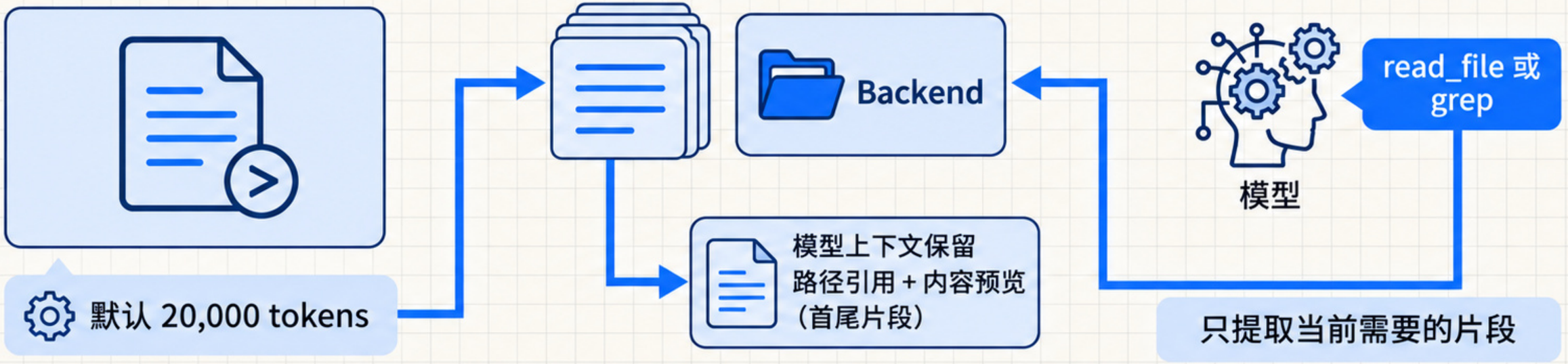
工具输出超过阈值

Step 2: 保存并替换

完整内容保存到 Backend

Step 3: 按需读回

read_file 或 grep



tool_token_limit_before_evict

⚠ 读回依赖保存成功，且文件仍可访问；不是永久归档保证。

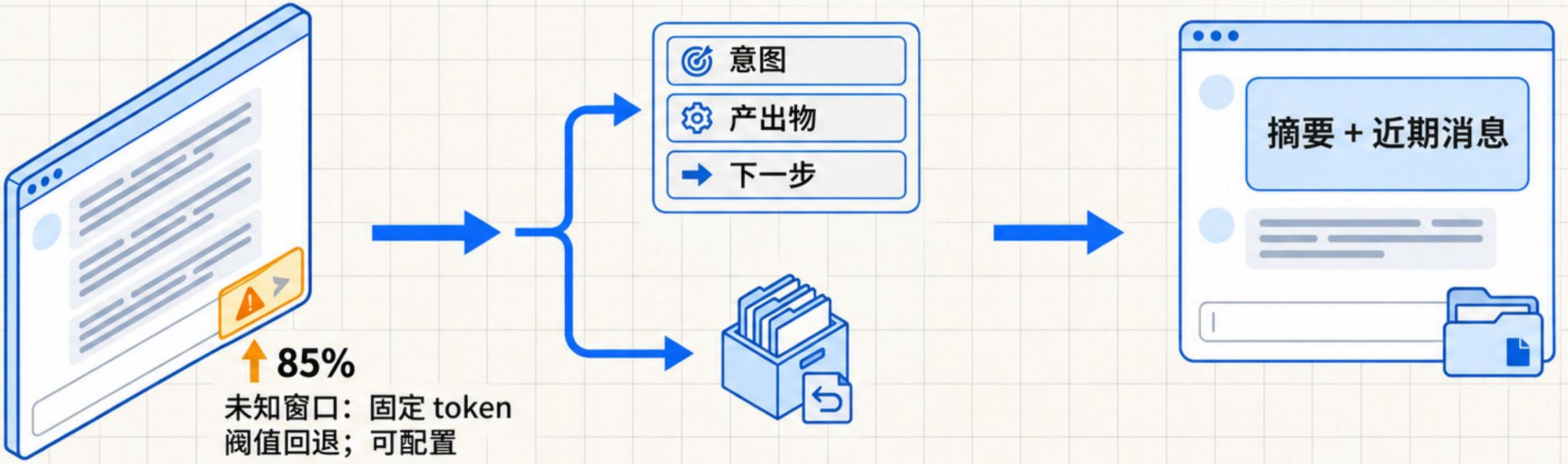
第二道防线：对话历史自动总结

压缩模型输入，不等于删除 State 历史

触发 / 已知模型窗口：默认约 85%

总结与保存 / 保存旧消息到 Backend

输入模型 / 摘要 + 近期消息

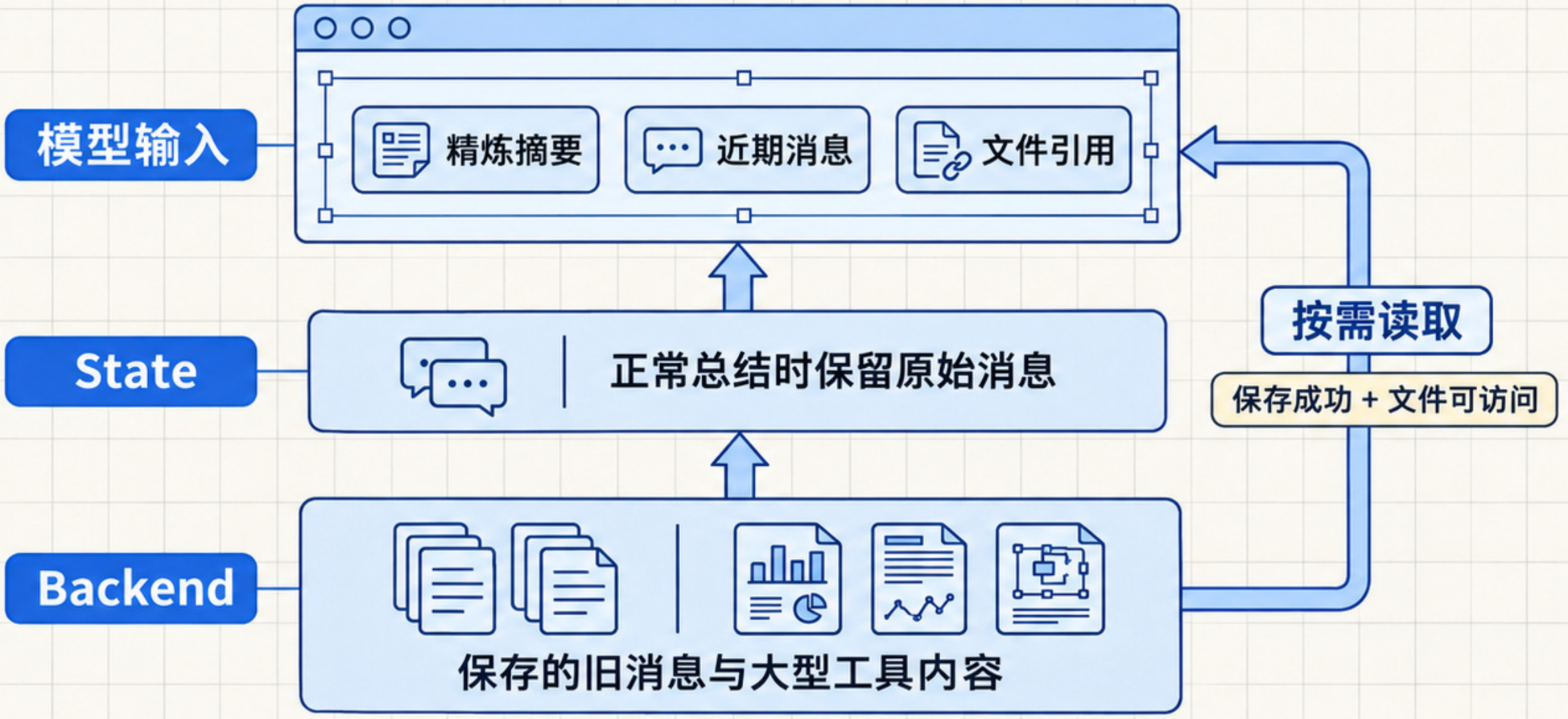


正常总结流程中，State 仍保留原始消息。

⚠ 文件读回仍取决于保存与访问是否成功。

工作记忆与完整记录，各有边界

区分模型输入、State 与 Backend



⚠ 文件是否持久，由 Backend 与存储配置决定；不能承诺任何记录都能永久读回。

上篇回顾

1. 结构化工作空间，让 Agent 按需存取资料
2. 七大工具：读写、编辑、删除、列举与检索
3. read_file 默认 100 行，按分页位置继续读取
4. grep 是字面量匹配，搜索可能被截断 ⚠
5. 卸载大型工具内容，总结旧对话，控制模型输入
6. State 历史与模型输入不同；读回依赖后端可用性 ⚠

下篇：可插拔存储后端

State · Filesystem · LocalShell · Store · Composite · Sandbox